



國立中央大學
National Central University

Autonomous Exploration and SLAM Mobile Robot

Department of Mechanical Engineering, Systems Division

Advisor: Prof. Hsien-Ting Chang

Author: Chao Po-Ju Hsu Tang-Wei

Date: 2025 June

1. Abstract

This thesis presents the design and implementation of an autonomous differential-drive mobile robot intended for indoor service tasks, such as transporting trays. The motivation arises from the need to reduce human effort and improve efficiency in tasks like food delivery or carrying objects in restaurants and domestic environments. To address this challenge, a differential-drive chassis was developed and integrated with a SLAM system for simultaneous localization and mapping, along with AMCL-based localization and a navigation stack for path planning and obstacle avoidance. The proposed system allows the robot to autonomously navigate to user-specified destinations while transporting items safely and reliably. Experimental tests in an indoor environment demonstrated that the robot could achieve point-to-point navigation and successfully complete tray-carrying tasks. These results validate the feasibility and effectiveness of applying standard SLAM and autonomous navigation frameworks to differential-drive service robots.

2. Preliminaries

2.1 Hardware

Host Computer

- Model: MSI Sword Laptop
- CPU: Intel Core i7-13620H (please confirm)
- GPU: NVIDIA RTX 4060 Laptop GPU
- CUDA Version: 12.8
- Memory: 32GB RAM

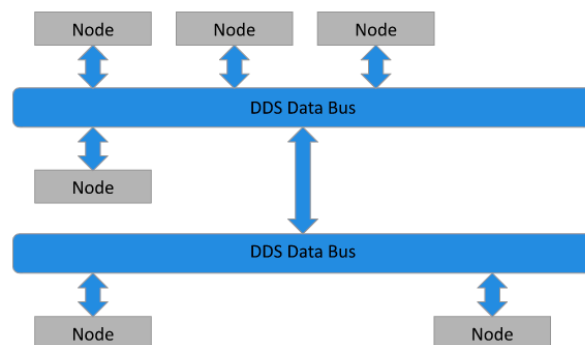
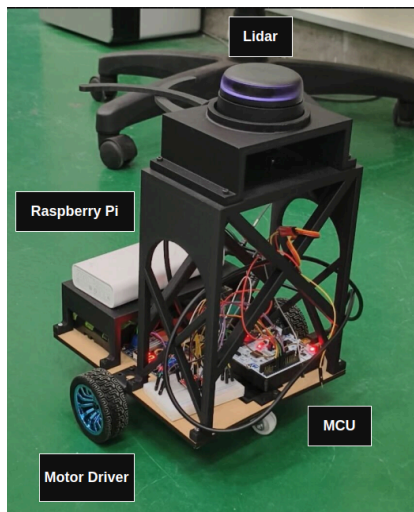
Vehicle (Mobile Robot)

- Compute Module: Raspberry Pi 4B
 - Processor: Broadcom BCM2711, Quad core Cortex-A72 (ARM v8) 64-bit @ 1.5GHz
 - Memory: 4GB LPDDR4
- LiDAR Sensor: SLAMTEC RPLIDAR A2M12
 - Scanning Frequency: 10 Hz typical
 - Max Scanning Radius: 12 meters
 - Angular Resolution: approx. 0.45°

- Microcontroller: STM32F407
 - Role: low-level motor and sensor control
 - Communication: UART link to Raspberry Pi
- Motor Driver: TB6612FNG

2.2 Software

In this project, we adopted ROS 2 (Robot Operating System 2) as the primary software framework for integrating and managing the autonomous vehicle's functions. ROS 2 is a widely used middleware framework built on Linux that facilitates modular robot software development, supporting a publish-subscribe architecture that allows different software components to communicate seamlessly. In ROS 2, each process or functional module is implemented as a node, enabling flexible and distributed system design. Communication between nodes is accomplished through topics, which provide publisher and subscriber interfaces for data exchange, and through services, which allow request-response interactions for more structured data handling. The ROS 2 system communicates with the STM32F4 microcontroller using a UART serial interface, enabling reliable command and sensor data transmission between the Raspberry Pi (high-level control) and the STM32 (low-level motor and sensor control). High-level algorithms, including SLAM, AMCL localization, and path planning, run on the Raspberry Pi, while the low-level motion control logic is executed on the STM32. This architecture allows for an efficient division of tasks between high-level autonomy and real-time actuation.



3. Methodology

3.1 SLAM

SLAM stands for Simultaneous Localization and Mapping, which refers to the technique of mapping an unknown environment while simultaneously using self-localization to explore it. The main SLAM methods include Gmapping, Hector, and Karto. The diagram below shows that Karto's algorithm effectively performs loop

closure, ensuring the map is closed. However, due to the small area of the simulated environment, we chose to use the Gmapping algorithm based on our test results.

3.1.1 Gmapping

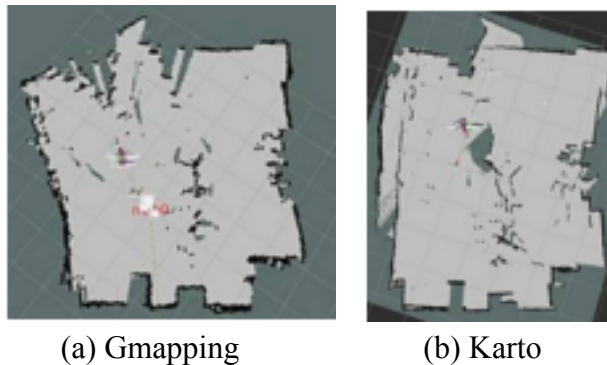
According to SLAM ToolBox, Gmapping is a method that integrates observation results with sensor data to create maps. It utilizes observation data from LiDAR or depth cameras, combined with information from encoders and an IMU (Inertial Measurement Unit), using particle filtering to estimate the most likely position and orientation of the robot. This allows for mapping of unknown environments. However, because this method involves spreading many points for calculations, it is less suitable for large areas with many features. It tends to perform better in indoor environments with more controlled scanning conditions.

3.1.2 Karto

Karto also utilizes sensor data and observation results for mapping, but unlike Gmapping, Karto incorporates a closure loop detection method to optimize the map. Figure 2 shows the comparison of scanning results using Gmapping and Karto. This approach reduces the accumulated errors that can occur during the mapping process. As a result, Karto is better suited for mapping larger environments compared to the other methods.

3.1.3 Hector

This algorithm does not require the use of encoders and can perform mapping in uneven environments. However, it needs a high update frequency.



3.2 Navigation

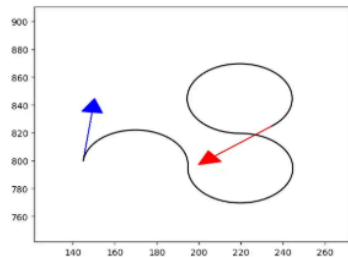
In this project, the navigation capabilities were implemented using the Nav2 framework, which is the successor to the ROS 1 navigation stack and supports modern ROS 2 architecture. Nav2 provides a modular and extensible navigation pipeline, including global planning, local planning (motion control), recovery behaviors, and costmap management. The navigation system enables the robot to plan collision-free paths, follow them reliably, and react to dynamic obstacles in the environment. Our implementation includes a customized path-planning module, a model predictive control algorithm for motion tracking, and a manual override function to allow operator intervention when needed. The following subsections describe each component in detail.

3.2.1 Path Planning — RRT* Plugin

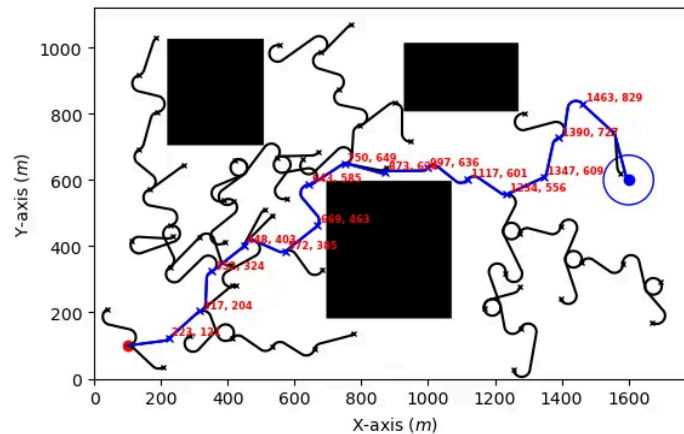
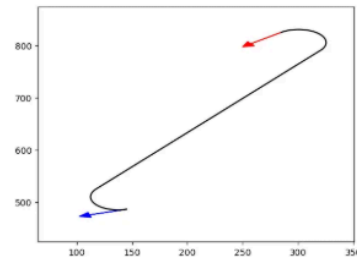
In this project, we developed a customized RRT* (Rapidly-exploring Random Tree Star) algorithm as the global path planner within Nav2. RRT* is an incremental sampling-based planning algorithm that extends the classical RRT by introducing a rewiring process, which enables asymptotic optimality of the planned path. The main advantages of RRT* include:

- Its ability to handle high-dimensional and complex spaces efficiently
- Probabilistic completeness, guaranteeing a solution if one exists
- Flexibility to adapt to dynamic and partially known environments
- Scalability to larger maps without a significant increase in planning time

CCC Curve



CSC Curve



By integrating this RRT* planner into Nav2, we achieve a global path planner that can continuously improve path quality while maintaining real-time performance suitable for indoor autonomous navigation.

3.2.2 Motion Control — MPPI

For local motion control and trajectory tracking, we implemented the Model Predictive Path Integral (MPPI) algorithm. MPPI is a sampling-based model predictive control method that evaluates multiple candidate trajectories by simulating

their costs over a short time horizon and selecting the best control input based on these predictions. The key strengths of MPPI include:

- Robust performance in the presence of model uncertainty
- Capability to handle nonlinear dynamics of the robot
- Direct incorporation of control constraints and obstacle avoidance into the optimization
- Real-time computation through parallel sampling of trajectories

By using MPPI as the local controller, the robot is able to follow the global RRT*-generated paths while reacting effectively to disturbances or unforeseen obstacles.

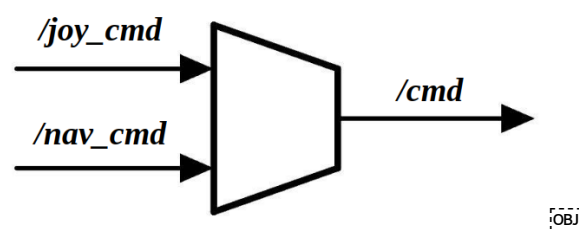


(a) Start Navigate

(b) Reach Goal

3.2.3 Manual Override – Twist

In addition to the autonomous navigation pipeline, we implemented a manual override mechanism to allow the operator to directly control the robot if needed. This is achieved by maintaining two separate command topics: `/joy_cmd` for joystick commands and `/nav_cmd` for autonomous navigation commands. A switching mechanism determines the current active command based on a selected mode value, effectively choosing which source to forward to the `/cmd_vel` topic. This design allows seamless switching between autonomous and manual teleoperation modes, enhancing flexibility and safety during deployment and testing.



3.3 Odometry

In this project, odometry estimation is achieved using the Range Flow-based 2D Odometry (RF2O) algorithm. RF2O is a scan-matching approach that uses raw 2D LiDAR range data to directly estimate the motion of the robot. Unlike conventional odometry methods that rely solely on wheel encoders, RF2O exploits the dense spatial and temporal information available in LiDAR scans to infer the robot's linear and angular displacements.

The fundamental idea of RF2O is to model the flow of range measurements over time, treating the changes in successive scans as a flow field similar to optical flow in images. By minimizing a cost function based on these flow constraints, the algorithm computes a highly accurate estimate of the robot's planar motion between consecutive scans.

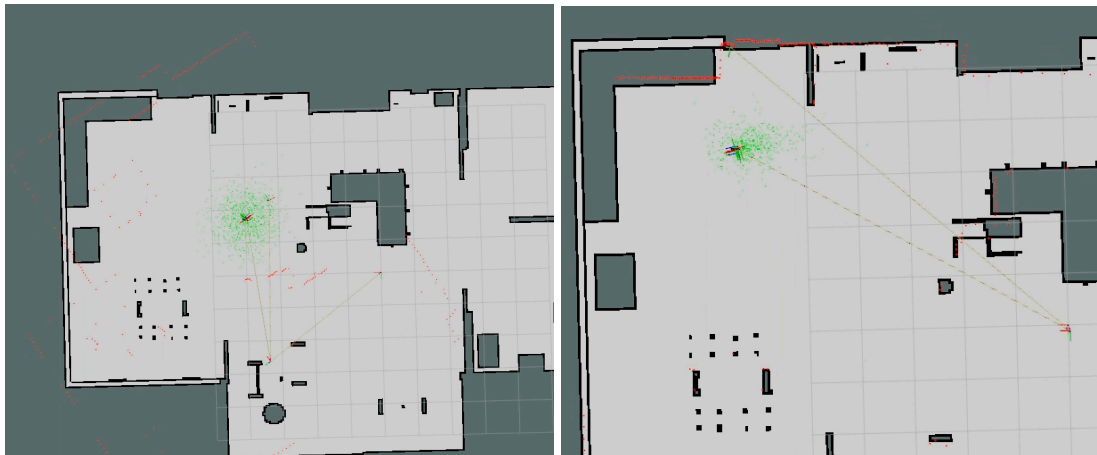
The main advantages of the RF2O method include:

- Robustness to wheel slip: because it is independent of wheel encoders, errors caused by slippage or uneven terrain are greatly reduced
- High precision: the scan-based approach provides millimeter-level accuracy under good LiDAR conditions
- No reliance on external markers: RF2O only needs the 2D laser scans, avoiding the need for visual landmarks or fiducials
- Lightweight and fast: the algorithm can run in real time on modest embedded hardware

In our system, RF2O acts as the primary source of odometry, providing reliable motion estimates that are fused with the global pose estimates from AMCL, supporting consistent localization and smooth navigation. This combination allows the differential-drive robot to operate robustly even under wheel slip or encoder errors.

3.4 AMCL

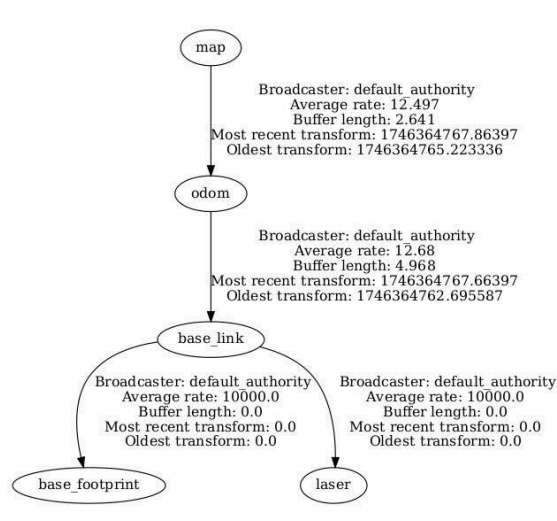
The Adaptive Monte Carlo Localization (AMCL) algorithm is employed in this project to provide robust pose estimation within the prebuilt map. AMCL uses a particle filter to maintain a probability distribution over the robot's possible positions, continuously refining this estimate by incorporating laser scan measurements and motion updates. One of the key advantages of AMCL is its ability to converge to the correct position even if the initial pose estimate is poor. As the robot moves and accumulates more sensor observations, the particle distribution gradually concentrates around the true location, effectively correcting for errors in the initial guess. This capability ensures that the robot can reliably localize itself within the environment, maintaining consistent and accurate navigation performance even when subject to initialization uncertainties.



3.5 ASLAM

4. Discussion

Throughout the development and testing of the autonomous differential-drive platform, several challenges were encountered. One of the most significant difficulties was related to the TF (transform) tree configuration. Because the LiDAR sensor was used as the primary source of odometry, any mistakes in setting up its frame relationships would cause discontinuities in the transform tree, leading to errors in localization and inconsistent system behavior. These interruptions in the TF broadcast affected the robot's ability to maintain a stable pose estimate, requiring repeated adjustments and validation of the TF tree definitions.



Full TF Tree

Another issue arose in the global planning subsystem. The global planner was implemented using an RRT* algorithm combined with Dubins path smoothing to generate feasible trajectories. However, since the local motion controller (based on the MPPI algorithm) was responsible for tracking and smoothing the path in real time, the Dubins smoothing became somewhat redundant. In practice, the MPPI controller ignored many of the finer Dubins-generated path features because it recomputed a

smoothed control trajectory on its own. As a result, incorporating Dubins path smoothing within the global planner did not yield significant benefits and could be considered unnecessary in this context. A simpler straight-line or piecewise linear global planner might have been more efficient when paired with a powerful local controller such as MPPI.

5. Conclusion and Future Work

For future improvements, we plan to enhance the SLAM capabilities of the system by making the entire mapping and exploration process fully autonomous. Currently, the robot relies on user commands to initiate mapping and navigation. To advance this, we propose incorporating a frontier-based exploration algorithm. Frontier exploration techniques enable the robot to identify unexplored regions of the environment (frontiers) and autonomously plan paths to explore and map these areas. By integrating such an algorithm with the existing SLAM and navigation stack, the robot could perform continuous, unsupervised exploration to build complete maps of unknown environments without human intervention. This advancement would significantly increase the system's level of autonomy and expand its applicability to more dynamic and large-scale scenarios.